

Programming and Algorithms

Aula 09: Estrutura de repetição indefinida while

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Aulas passadas

- Estruturas de seleção simples e compostas
  - if, else e elif
- Formatação e operações com strings
  - f-strings
- Funções
  - importadas
  - definidas
  - escopo de variáveis
- Treinamentos 1 e 2 da plataforma beecrowd

Programa da Aula

- Estrutura de repetição para controle de fluxo
- Estrutura de repetição indefinida (while)
- Variáveis contadoras, repetidoras e de sinalização booleana para controle do while
- O que é laço infinito e suas consequências
- Estruturas de repetição combinadas com estruturas de seleção

Motivações

- Repetição
  - Repetir um cálculo várias vezes em tempos baixíssimos é uma das vantagens dos computadores sobre os humanos. Então é bom aprender como fazer.

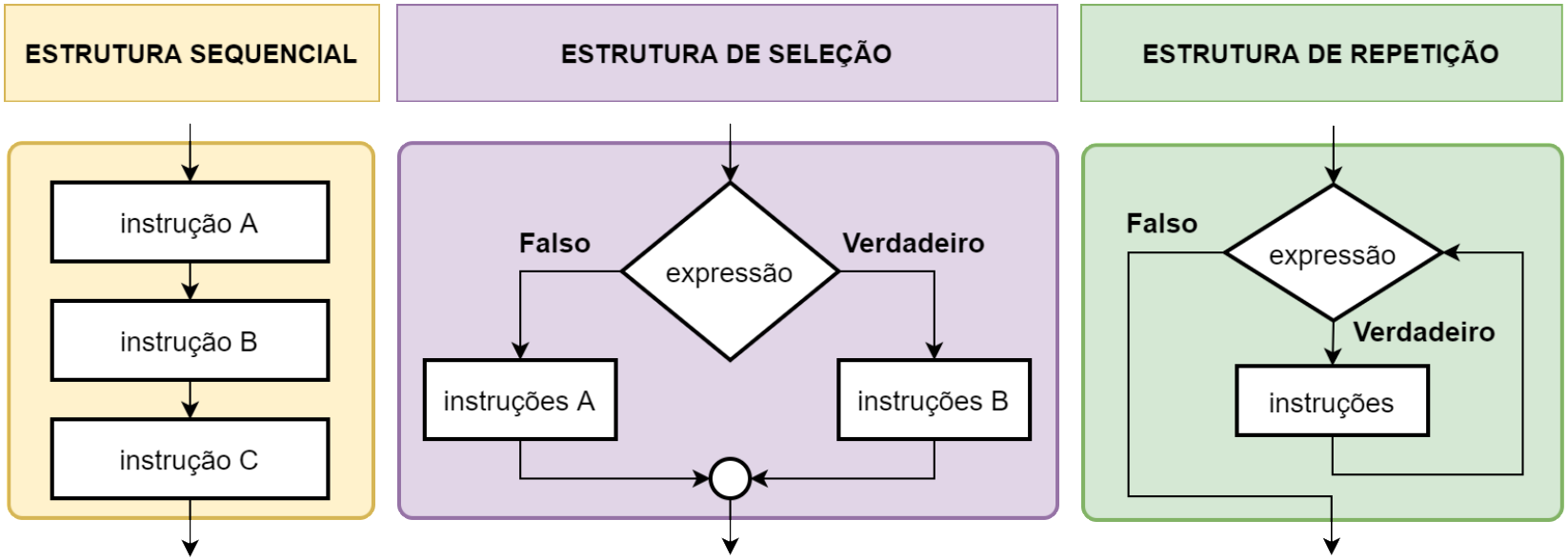


Figura 7.1: Representação em fluxograma das estruturas básicas de controle de fluxo.
Fonte: Elaborado pelo autor.

Quando utilizar?

- Alguns sinônimos para estruturas de repetição:
  - Estrutura de iteração
  - Laços
  - Loops
  - Malhas de repetição
- Pergunta importante:
  - Quando utilizar?

    Quando o programa pedir/precisar da repetição de uma sequência de instruções várias vezes

Problema

Criar um programa que exiba os cinco primeiros números naturais positivos.

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

```
x = int(input('Valor: '))
print(x + 1)
print(x + 2)
print(x + 3)
print(x + 4)
print(x + 5)
```

```
x = int(input('Valor: '))
x += 1
print(x)
x += 1
print(x)
x += 1
print(x)
x += 1
print(x)
x += 1
print(x)
```

Qual o problema dessas abordagens?

Estruturas de repetição

- Se você quiser mostrar os 50 ou 100 números na sequência de x, basta replicar um comando 50 ou 100 vezes, o que exigiria um número grande de linhas de código.
- Embora seja possível resolver estes exemplos com o que aprendemos até agora, há outros que são inviáveis.
- As estruturas de repetição aparecem para nos auxiliar a resolver este tipo de problema.
- Vamos aprender mais sobre isso.

Estruturas de Repetição

Exemplo: O programa quer exibir uma sequência de números naturais limitada incluindo os valores inicial e final. Por exemplo, valor inicial 5 e valor final 12. A saída deve ser todo o intervalo [5,12]. E por que não é possível resolvê-la com o que sabemos?

R. Pois não há como prever a quantidade de valores exibidos pois isso dependerá dos valores fornecidos pelo usuário que só será descoberto quando o programa estiver em execução

Tipos de estruturas de repetição

1. Estruturas de repetição com uma quantidade de repetições indefinida:

É impossível saber quantas repetições serão executadas até que o loop comece

2. Estruturas de repetição com uma quantidade de repetições definidas

É possível determinar quantas repetições serão executadas antes do loop começar

Estrutura de Repetição While

É uma estrutura de repetição com uma quantidade de repetições **indefinida**

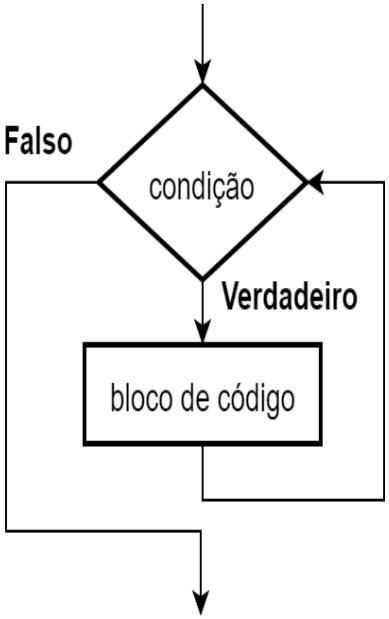
Sintaxe:
```
while < condição >:
    bloco de código
```

Note a indentação padrão que indica o início do bloco de código.

Como já vimos as instruções contidas no bloco do código só serão executadas caso a condição resulte em **True.** Caso a condição resulte em **False**, o bloco de código é ignorado e o programa segue em frente.

O comando While

- Há, no entanto uma diferença fundamental com o comando while:
- Enquanto sua condição resultar em True o bloco de código voltará ao início para ser executado até que a condição resulte em **False.**



Enquanto a condição resultar em True, o bloco de código será executado.

Quando a condição resultar em False, o fluxo é deslocado para a próxima instrução fora do *loop*.

Figura 7.2: Representação em fluxograma da estrutura de repetição `while`.
Fonte: Elaborado pelo autor.

O comando While

Resolvendo o problema do início da aula, rode no python tutor.

```
x = 1
while x <= 5:
    print(x)
    x +=1
print('Tchau !')
```

Também no python tutor. Para fornecer os 5 primeiros naturais **posteriores** ao valor dado na entrada

```
1 x = int(input('valor: '))
2 qtd_exibidos = 0
3 while qtd_exibidos <= 5:
4     x+=1
5     print(x)
6     qtd_exibidos += 1
7 print('fim')
```

O comando While

```
inicio = int(input('Início: '))
fim = int(input('Fim: '))
x = inicio
while x <= fim:
    print(x)
    x += 1
```

Note que o incremento está no final do bloco. Ela é a varíavel que vai controlar a execução do loop. Como interpretar o loop?

a) Primeiro ele executa a instrução. Neste caso print(x)

b) Depois incrementa a variável x e volta ao início para testar a condição até que ela resulte em False para seguir em frente no programa.

Contadores

- Para construir laços de repetição utilizamos muitas vezes uma variável contadora (contador) que é útil para contar quantas vezes o bloco de intruções do laço foi executado e/ou controlar quantas vezes ele será executado.
- Uma variável contadora é incrementada/decrementada por um valor constante.

Contadores

Vamos escrever um programa de tal forma que o último número a ser impresso seja informado pelo programador.

```
fim = int(input('Digite o último número a ser impresso: '))
x = 1
while x <= fim:
    print(x)
    x += 1
```

Contadores

```
fim = int(input('Digite o último número a ser impresso: '))
x = 1
while x <= fim:
    print(x)
    x += 1
```

Dentro do bloco do while: x será impresso e a seguir atualizado com o próximo valor inteiro. Quando fazemos isso dentro de uma estrutura de repetição, estamos contando. Ou seja, x é um contador que é uma variável utilizada para contar o número de ocorrências de um determinado evento; neste caso, o número de repetições do While que satisfaz à demanda do problema.

Contadores

O código abaixo usa a variável contadora apenas pra contar quantas vezes o bloco de instruções do laço foi executado. Ela não influencia o número de repetições do laço e não é usada como variável de controle.

```
executa = input('execute o bloco do laço: ')
contador = 0
while executa == 'sim':
    contador = contador +1
    executa = input('execute o bloco de laço novamente: ')
print(f'O bloco de laço foi executado {contador} vezes')
```

```
execute o bloco de laço novamente: sim
execute o bloco de laço novamente: sim
execute o bloco de laço novamente: nao
O bloco de laço foi executado 3 vezes
```

Contadores

O código abaixo usa uma variável contadora para controlar a quantidade de vezes que o bloco de instruções do laço será executado.

A variável contadora **influencia o número de repetições do while**, ou seja, é usada também como uma variável de controle do laço.

Contagem regressiva

```
#Contagem regressiva
contador = 10
while contador > 0:
    print(contador)
    contador -= 1
print('Lançar foguete! ')
```

Acumuladores

- Uma variável acumuladora é usada para acumular valores que, em geral, não são constantes.
- Em programas para o calcular o total de uma soma, por exemplo, precisaremos de acumuladores (ou variáveis acumuladoras).
- A diferença entre um contador e um acumulador é que: **nos contadores o valor adicionado é constante enquanto que nos acumuladores é variável.**
- Vejamos o seguinte programa que calcula a soma de 10 números

Acumuladores

Programa que calcula a soma de 10 números.

```
n = 1
soma = 0
while n <= 10:
    x = int(input(f'Digite o {n} número'))
    soma = soma + x
    n += 1
print(f'Soma: {soma}')
```

Neste caso, a variável soma é o acumulador e n é o contador

Acumuladores

Crie um programa que receba como entrada preços de itens comprados em um mercado por um dado cliente. No final, o programa deverá exibir o total da compra. Para informar que não há mais itens a serem comprados, o cliente deve digitar o valor -1.

```
total = 0  # variável acumuladora

preco = float(input('Preço do item: '))

while preco != -1:
    total = total + preco
    preco = float(input('incluir outro preco: '))

print(f'O total da compra deu: R$ {total:.2f}')
```

Variáveis acumuladoras

Crie um programa que receba como entrada o crédito de um cliente e depois os preços dos itens comprados em um mercado por este cliente. O programa deverá parar de solicitar novos itens quando o crédito disponível for insuficiente para pagar por um deles. Ao final, o programa deve exibir o total da compra e o crédito restante.

Variáveis acumuladoras

```
credito = float(input('Seu crédito: ')) # variável acumuladora
total = 0 # variável acumuladora

preco = float(input('Preço do item: '))
while credito >= preco:
    total = total + preco
    credito = credito - preco
    preco = float(input('Preço de mais um item: '))

print(f'Total da compra: R$ {total:.2f}')
print(f'Crédito restante: R$ {credito:.2f}')
```

```
Seu crédito: 100.00
Preço do item: 50.00
Preço de mais um item: 10.00
Preço de mais um item: 25.00
Preço de mais um item: 16.00
Total da compra: R$ 85.00
Crédito restante: R$ 15.00
>>
```

Rode no python tutor

Variáveis acumuladoras

Exercício 6 (apostila): Refaça o programa anterior de modo que todo produto inserido pelo usuário seja numerado sequencialmente, iniciando em 1, e que o programa exiba a mensagem “Compra do ítem X negada!”, após a leitura do item que extrapolar o valor do crédito, onde X é o número do item. Acrescente as instruções necessárias para ao final exibir a quantidade de itens que puderam ser comprados.

Variável Flag Booleana

As vezes criamos laços em que a condição é dada por uma variável com valor booleano. Então, a variável para controlar o loop é uma flag booleana, pois ela sinaliza se o loop deve ser encerrado ou não. Também é possível criar uma condição com mais de uma flag booleana.

Flag booleana

```
total = 0
quero_comprar = True

while quero_comprar:
    preco = float(input('Preço: '))
    total = total + preco
    opcao = input('continuar comprando (s/n)? ')
    if opcao != 's':
        quero_comprar = False

print(f'Total da compra: R$ {total:.2f}')
```

Loop infinito

É uma estrutura de repetição que nunca é encerrada, ou é encerrada por conta de um erro eventual no programa ou por algum comando externo. Normalmente são gerados por erros de lógica

Teste as codificações no editor e shell e arrume para que a execução seja encerrada após a exibição dos pares no intervalo [0, 10].

```
n = 0
while n <= 10:
    print(n)

print('Acabou!')
```

```
n = 0
while n >= 0:
    print(n)
    n += 2
print('Acabou!')
```

```
n = 0
while n <= 10:
    if n % 2 == 0:
        print(n)
    n += 1

print('Acabou!')
```

```
n = 0
while n <= 10:
    print(n)
    n +=2
print('Acabou')
```

Combinação de estruturas de controle de fluxo

- É comum utilizarmos a combinação de estruturas de repetição com estruturas de seleção na resolução de problemas.
- Um exemplo é a criação de uma função que recebe como argumento um número natural n e devolve um valor booleano indicando se ele é primo. Lembrando: um número primo possui exatamente dois divisores naturais, o número 1 e ele mesmo.

Combinação de estruturas de controle de fluxo

```
def primo(n):
    qtd_de_divisores = 0
    divisor = 1
    while divisor <= n:
        if n % divisor == 0:
            qtd_de_divisores += 1
        divisor += 1

    if qtd_de_divisores == 2:
        return True
    else:
        return False
```

Exercícios

1. Crie um programa que exiba todos os números inteiros de 100 até 200 em ordem crescente. Depois crie outro programa que exibe de 200 até 100 em ordem decrescente.

2. Crie um programa que leia um número natural n dado pelo usuário e exiba só os n primeiros pares a partir de 0. Por exemplo, n= 6 será exibido 0, 2, 4, 6, 8, 10

3. * Crie um programa que solicite ao usuário dois números naturais x e y. Ele deve exibir o quociente da divisão inteira de x por y sem usar os operadores de divisão e multiplicação. Por exemplo, o quociente da divisão inteira de x por y é dado pela quantidade de vezes que y pode ser subtraído de x sem que x se torne negativo.

Exercício 4: Crie um programa que peça letras como entrada, uma por vez, até que seja lida a letra ‘x’ ao final. O programa deve exibir a quantidade de letras lidas sem contabilizar o ‘x’.

Soluções:

1. Usando funções:

- Ordem crescente

```
def exibir_numeros():
    numero = 100
    while numero <= 200:
        print(numero)
        numero += 1
```

- Ordem decrescente

```
def exibir_numeros_2():
    numero = 200
    while numero >= 100:
        print(numero)
        numero -= 1
```

Soluções:

2. Usando função e o bloco try... except (que não veremos)

```
def primeiros_pares(n):
    numero = 0
    contador = 0

    while contador < n:
        print(numero)
        numero += 2
        contador += 1

try:
    n = int(input("Digite um número natural n: "))
    if n < 0:
        print("Por favor, insira um número natural positivo.")
    else:
        primeiros_pares(n)
except ValueError:
    print("Por favor, insira um número válido.")
```

Soluções:

3

```
dividendo = int(input("Dividendo: "))
divisor = int(input("Divisor: "))
quociente = 0
x = dividendo
while x >= divisor:
    x -= divisor
    quociente += 1
div_int = x
print(f'a divisão inteira de {dividendo} / {divisor} é {quociente}')
print(f'o resto da divisão inteira de {dividendo} / {divisor} é {x}')
```

Soluções:

4

```
quantidade_letras = 0
letra = ""

while letra != 'x':
    letra = input("Digite uma letra (ou 'x' para sair): ")
    if letra != 'x':
        quantidade_letras += 1

print(f"Você digitou {quantidade_letras} letras (excluindo 'x').")
```

FIM